Refactorización del script en funciones reutilizables (Bloque 6) / Tu script, ya refactorizado como módulo del repositorio

✓ **Hecho:** Ver

📘 **UNIDAD U1.3** · Python aplicado para integración con IA

Tu script, ya refactorizado como módulo del repositorio

Cierra este bloque con un momento para mirar hacia atrás. Ya no tienes la función única y algo apretada que construiste en el **Bloque 5**: tienes un módulo dividido en funciones, cada una con una responsabilidad clara, y con el mismo resultado exacto de siempre.

Antes de avanzar al **Bloque 7**, vale la pena entender bien qué cambió con este trabajo, y qué es lo que todavía falta, porque ambas cosas explican por qué el orden de esta unidad es el que es.

> Esta lección no te pide ninguna acción nueva sobre tu código: ya hiciste el trabajo en la lección anterior. Lo que sigue es, más bien, un momento de revisión, para que llegues al **Bloque 7** con una idea clara de sobre qué exactamente vas a construir el manejo de errores.

⚖️ Qué tienes ahora, comparado con el Bloque 5

En el **Bloque 5**, tu módulo era una sola función que hacía todo: armaba la **URL**, enviaba la petición, procesaba la respuesta, y devolvía el dato. Leerla de principio a fin significaba seguir cuatro tareas distintas mezcladas en un mismo bloque de código.

Ahora esa misma lógica está repartida en varias funciones pequeñas, cada una con un nombre que

describe justamente lo que hace, y una función principal que simplemente las llama en orden. Nada de lo que tu módulo produce cambió; lo que cambió es qué tan rápido se entiende, y qué tan fácil es tocar una sola parte sin arriesgar el resto.

Piensa en la diferencia como la de leer una receta escrita en un solo párrafo corrido, contra leerla dividida en pasos numerados. El contenido es el mismo, pero encontrar un paso específico, o repetir solo uno de ellos, es mucho más rápido en la segunda versión.

Dicho en código, de forma esquemática (ya viste el detalle completo en la lección anterior), la diferencia se ve así:

Antes (Bloque 5):

```
def obtener_dato_de_prueba(pregunta):
    # arma la URL, envía la petición, procesa la respuesta
    # y extrae el dato, todo junto en un mismo bloque
    ...
```

Después (Bloque 6):

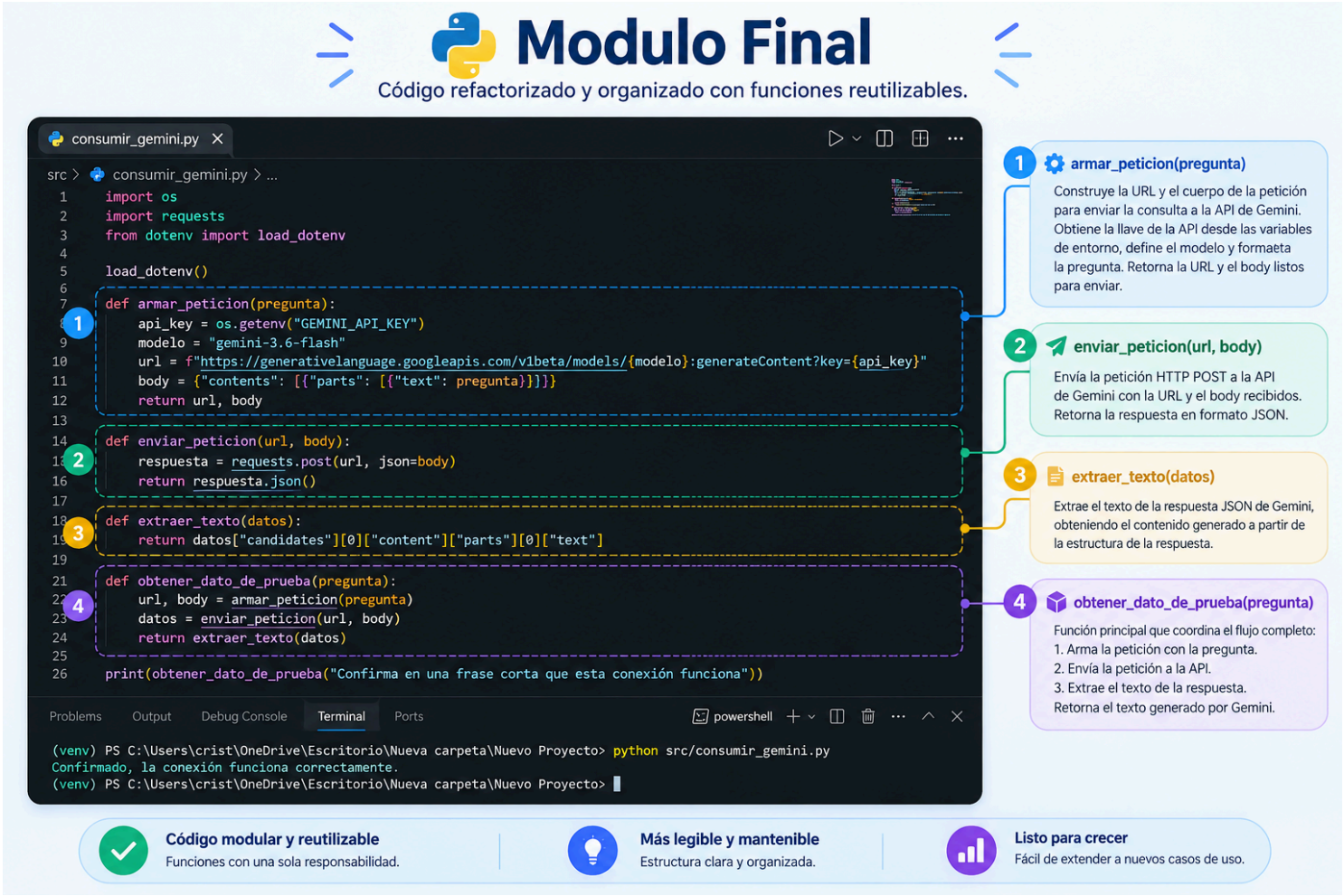
```
def obtener_dato_de_prueba(pregunta):
    url, body = armar_peticion(pregunta)
    datos = enviar_peticion(url, body)
    return extraer_texto(datos)
```

🚀 Por qué esta forma de organizar el código importa más adelante

El beneficio no es solo teórico. Piensa en la función que procesa la respuesta y extrae el dato: ahora que existe por separado, la puedes reutilizar tal cual el día que necesites procesar una respuesta parecida mientras consumes otro servicio, sin tocar nada de cómo se arma esa nueva petición.

Y si en el futuro necesitas cambiar cómo se arma la URL (por ejemplo, si el proveedor de un servicio actualiza su forma de identificar recursos), ese cambio queda contenido en una sola función, sin obligarte a revisar el resto del módulo para asegurarte de que nada más se vio afectado.

Hay también un beneficio que se nota más cuando el código lo lee alguien más, no solo tú. Un compañero que entra por primera vez a tu módulo puede, con solo mirar los nombres de las funciones y el orden en que se llaman, entender el flujo completo sin tener que leer el detalle de cada una. Eso es, en buena parte, lo que la rúbrica de calidad de código busca reconocer.



✨ Cómo se ve, en general, un módulo bien organizado

Más allá de este módulo en particular, conviene quedarte con algunas señales generales que indican que un código está bien organizado, porque las vas a seguir usando durante el resto del programa:



Una sola responsabilidad

Cada función hace una sola cosa, y esa cosa se puede describir en una frase



Nombres claros

Los nombres explican el propósito, sin necesidad de leer el código de la función para



Funciones breves

Ninguna función es demasiado larga, si te cuesta ver toda una función en pantalla sin



Función principal

Existe una función principal que orquesta, en vez de que toda la lógica esté repartida sin un

en una frase corta, sin usar la palabra "y" varias veces.

función para entender qué hace.

pantalla sin desplazarte, es una señal de que probablemente contiene más de una responsabilidad.

repartida sin un punto claro de entrada.

Vale la pena que revises tu propio módulo contra estas cuatro señales antes de seguir. Si alguna no se cumple todavía, este es el momento de ajustarla, antes de que el **Bloque 7** agregue una capa más de código encima.

Si alguna señal no se cumple todavía: no hace falta rehacer todo el módulo, solo ajustar la parte puntual.

> Si una función sigue haciendo más de una cosa, sepárala siguiendo el mismo patrón de tres funciones que ya usaste (armar, enviar, extraer).

> Si un nombre quedó vago, renómbralo: cambiar solo el nombre de una función no afecta su comportamiento, así que es un ajuste seguro.

> Si una función quedó demasiado larga, busca dentro de ella un paso que se pueda describir en una frase corta y aislarlo en una función nueva.

> Y si no hay una función que orqueste claramente a las demás, identifica cuál debería ser (normalmente la que ya tenía el nombre original) y redúcela a solo llamar a las otras en orden.

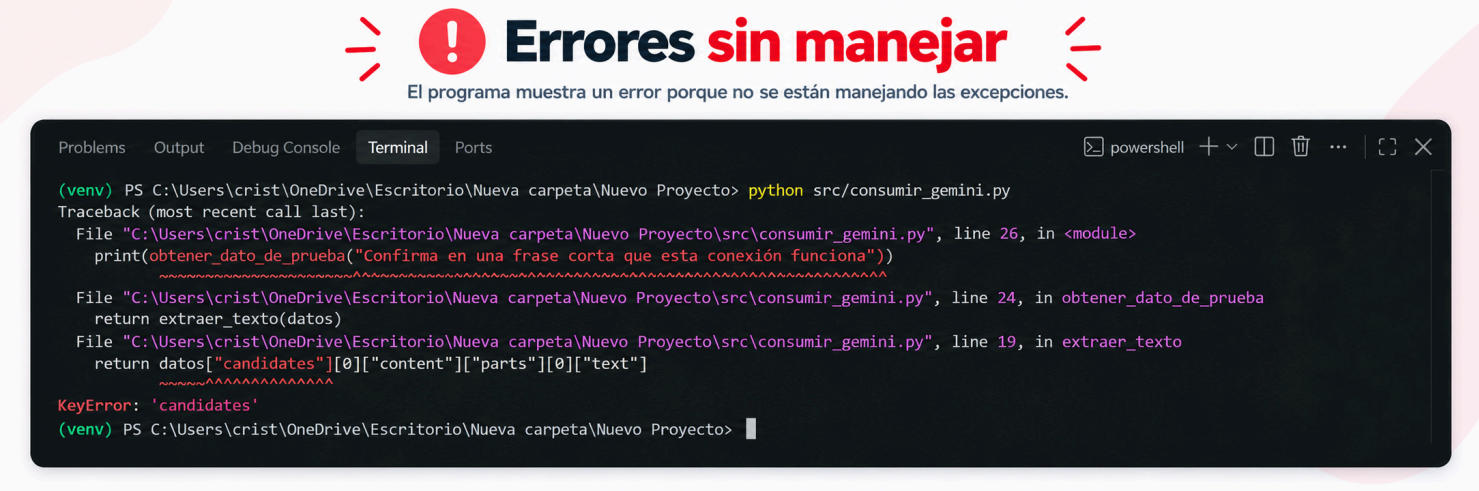
⚠ Lo que todavía no tiene tu módulo

> ⚠ **Todavía no maneja errores**
>
> Con todo esto, **tu módulo sigue siendo**, en un sentido importante, **incompleto**: no maneja ningún error todavía. Si el servicio no responde, o si la respuesta viene en un formato distinto al esperado, tu módulo se va a detener sin control, exactamente igual que en la demostración del Bloque 5.

Todavía no valida lo que recibe

Tampoco valida todavía lo que recibe: si alguien llama a tu función con un dato que no tiene sentido (un identificador vacío, o de un tipo distinto al esperado), el módulo va a intentar seguir adelante de todas formas, y el error va a aparecer más tarde y de forma menos clara de lo que debería.

Eso no es un descuido de este bloque, **es intencional**: primero se organiza el código que ya funciona, y solo después se blinda contra lo que puede salir mal. Meter manejo de errores antes de tener las funciones bien separadas suele terminar en código más confuso, no menos, porque estarías intentando anticipar fallas sobre una estructura que todavía no está clara.



Siguiente etapa

Bloque 7 · Excepciones y validación

El **Bloque 7** completa exactamente eso: vas a ver qué hacer cuando algo falla, vas a revisar en detalle la rúbrica de calidad de código que evalúa tu módulo, y vas a incorporar manejo de excepciones y validación de entradas a esta misma versión que acabas de refactorizar, el paso que cierra formalmente el entregable de esta **unidad**.

Última modificación: jueves, 24 de septiembre de 2026, 15:27